UNSTRUCTURED MESHING ASSIGNMENT

Mech 588 – Winter 2023 UBC Mechanical Engineering

Christian Rowsell
40131393

The following document describes the overall procedure used in writing an unstructured solver for the 2D, unsteady convection equation. Each step will be outlined, including testing, data structures to organize everything, code validation, and finally a description of the final problem.

1. Write a verification plan. Be sure to include tests for all of the key parts of your code, as well as global tests to confirm that your code correctly solves the PDE. Turn in your verification plan as part of your report, but you do not need to submit the results of your verification tests; if your code doesn't pass a reasonable set of verification cases, it won't solve the final problem, either.

   A test file was written which would take care of all of the testing for this code. The makefile was configured in such a way that all of the tests would be performed upon compilation of the code. If test happened to fail, it would let the user know how many tests failed, and which tests failed. A snippet of what this looks like for all of the tests being passed, and a variety of tests being failed can be seen below.

```
roject 2 - Unstructured Wave Equation$ make run
g++ -g -c src/test.cc -o src/test.o
g++ -g src/test.o src/mesh.o src/field.o -o test
./test


*********************************************
           TESTING
*********************************************
Vertices passed
Cell area passed
Edge length passed
Norm passed
Midpoint passed
Centroid passed
Gradient calculation passed
Gradient indices passed
Upwind cells succesfully tested for horizontal velocity test case
Upwind cells succesfully tested for vertical velocity test case
Edge flux calculation passed
Flux integration passed
L2Norm passed
*********************************************
           TESTING PASSED
*********************************************

Tests succesfully ran in 0.00000000 seconds
g++ -g src/mesh.o src/main.o src/field.o -o run
```

Figure 1: Example of All Testing Passed Message

```
 2 - Unstructured Wave Equation$ make run
g++ -g -c src/test.cc -o src/test.o
g++ -g src/test.o src/mesh.o src/field.o -o test
./test


*****************************************
           TESTING
*****************************************
Error reading mesh Y coordinate at vertex 0
Cell area passed
Edge length passed
Norm passed
Midpoint passed
Centroid passed
Gradient calculation passed
Gradient indices passed
Upwind cells succesfully tested for horizontal velocity test case
Error calculating dot product of edge norm and velocity for vertical velocity at edge 2
Edge flux calculation passed
Flux integration passed
L2Norm passed
*****************************************
        2 TESTS FAILED
*****************************************

Tests succesfully ran in 1.00000000 seconds
g++ -g src/mesh.o src/main.o src/field.o -o run
```

Figure 2: Example of when Testing Fails

In Figure 2, the testing for reading vertex coordinates, and calculating the dot product of the edge normal vector and the velocity vector failed. Having messages such as this during compilation allow the user to quickly determine where to look in the code, and points them toward the exact error. This saves on time when adding functions to the code, and helped with managing complexity as the code got longer.

A total of 13 tests were added to the test file, and the time step function was tested using a different method. Each of these tests will be discussed below:

Geometric Tests:

For the geometric tests, which include reading mesh vertices, calculating cell area, calculating edge length, calculating edge normal vector, calculating edge midpoint, and calculating cell centroid, a .mesh file based on the mesh used in the analytical calculation was used. This mesh can be seen below.

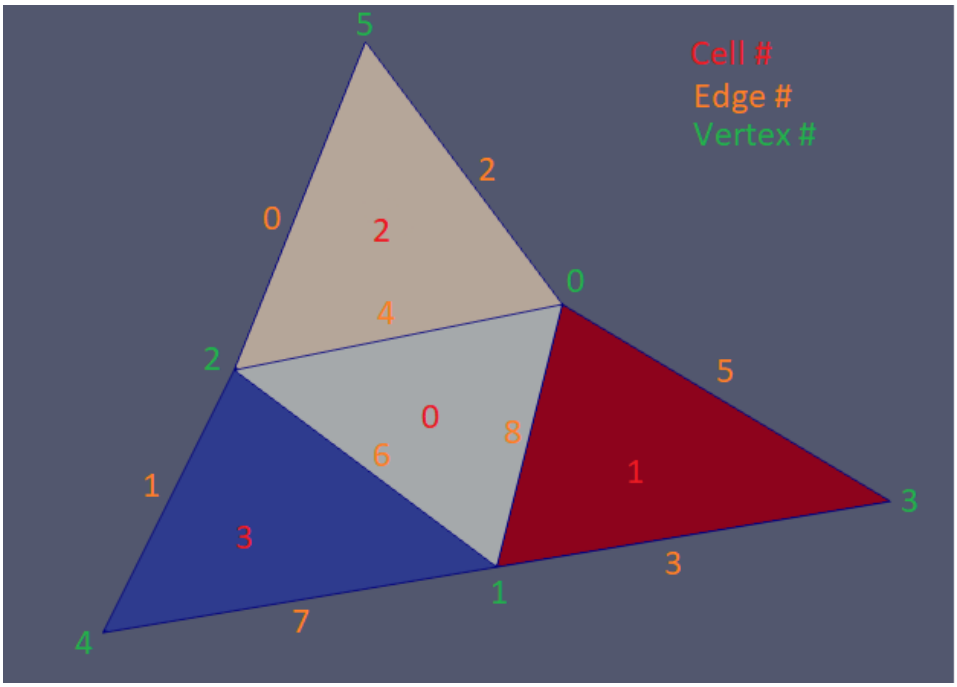


Figure 3: Test Mesh

This mesh allows for easy verification of cell connectivity data. It is simple enough that one can quickly determine a variety of information such as cell neighbors, cell edges, and cell vertices. Hand calculations were done to calculate information such as cell area, and then a MATLAB script was written which would calculate this information for each cell. The information from this MATLAB script was stored in a variety of test files, which were read to ensure the values were properly calculated when the testing script was run. In addition to explicitly testing that values were properly calculated based on the given mesh, a variety of other sanity checks were performed, that could be applicable to any of the mesh. For example, it was checked that no cell areas were negative, and that no edge lengths were negative.

For the cell gradient, a linear temperature field was imposed for the initial condition, and the gradient was calculated based on this temperature field. If the gradient calculation was properly constructed, it should reproduce the exact solution for a linear temperature field. The temperature was set to be T=5x+6y. The gradient should give results of exactly 5 for the gradient w.r.t. x, and exactly 6 for the gradient w.r.t. y. This is true regardless of mesh, and can be tested for each mesh.

The gradient indices checks that the proper indices were used in the gradient calculation. In the gradient calculation, the values of 3 other cells must be used to calculate the gradient in the cell of interest. We can call the cell of interest cell i, while the other cells used in the reconstruction can be named a, b, and c. The most obvious choice of cells for a, b, and c would be the neighbors of cell i. However, if cell i lies on the boundary, then it will not have 3 neighbors. If 1 edge of cell i sits on the boundary it will only have 2 neighbors, while if 2 edges of cell i sits on the boundary, it will only have 1 neighbor (as is the case for cells 1, 2 and 3 in Figure 3). As such, some logic must be used to get around this. There are two options available:

1. Pick from the list of cells that share a vertex with cell i. This option is good, because it provides many cells to choose from.
2. Use neighbors of neighbors of cell i. For example, in Figure 3, cell 3 only has 1 neighbor, which is cell 0. In this case, we can use the 2 neighbors of cell 0, and get a total of 3 cells to use.

Option 2 was selected, for these calculations. It is important to ensure that none of the neighbors of the cell includes the boundary (an index of -1 is placed in the list of cell neighbors if one of the neighbors is boundary). It is also important to ensure that none of the cells used in the gradient calculation repeat. This test checks that both of these conditions are true, and can be used for any of the provided mesh files.

For the upwind cells calculation, two tests are performed. The first test checks that the dot product of the edge normal and velocity vectors is properly calculated, while the second test checks that the upwind cell was properly determined. To perform these checks, a second small test mesh was created.

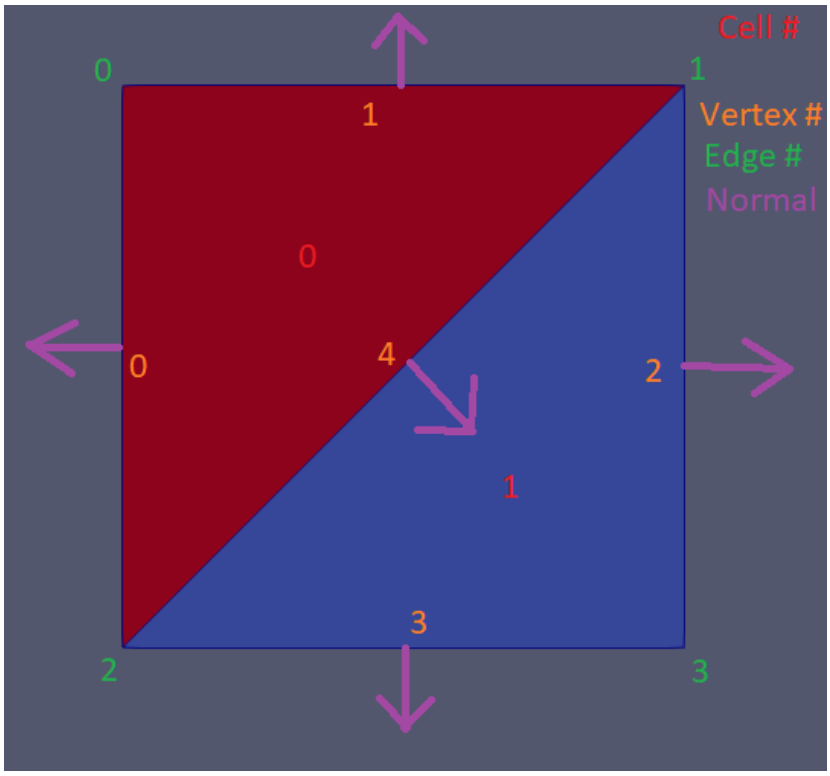


Figure 4: Upwind Calculation Test Mesh

Two tests were then performed. The first imposed a velocity with magnitude 1 in the positive x (from left to right) direction. In this test, the dot product was expected to be 0 for edges 1 and 3, -1 for edge 0, and 1 for edge 2. For edge 4, cell 0 was expected to be upwind, while cell 1 was expected to be downwind. For test 2, a velocity with magnitude 1 in the positive y direction (from bottom to top) was imposed. Now the dot product was expected to be 0 for edges 0 and 2, 1 for edge 1, -1 for edge 3, and for cell 4, 1 was expected to be upwind, while 0 was expected to be downwind.

For the edge flux calculation test, the temperature field was set to be the same as that found in the analytical part of this assignment, and the flux at edge 8 in Figure 3 was calculated. It was calculated for two test cases. The first had an imposed velocity with magnitude 2 in the positive x direction, while the second involved an imposed velocity with magnitude 2 in the negative y direction. The flux at the edge was then validated using hand calculations, and the MATLAB script.

The flux integration test takes advantage of the fact that the divergence of the flux can be related to the surface integral of the flux using divergence theorem. In this case, a temperature field of 1-x was

imposed on the field, while a velocity of 1 in the positive x direction was used. The divergence of the flux based on these conditions is expected to be 1 (or -1 depending on which side of the equation we place things on). As such, we expect a total flux integral of 1 for all of the interior points (the boundary conditions ensure that this is not true for the cells on the edge).

The L2Norm of a vector was simply tested by hand calculating the L2Norm of an arbitrary vector, and ensuring the results match. Finally, the time stepping was tested using the actual problem given in the problem description. In this case, if the time stepping was properly imposed, the initial condition should be moved to the x-axis after T=0.5, and it should have returned back to its original position after T=2.

Finally, the time stepping was tested using the problem assigned. A total of 20 vtk files were saved, at times evenly distributed through the total simulation time (the time for 1 period, or 2 units of time). These vtk files can be viewed in Paraview, in sequential order to ensure that the proper motion is seen.

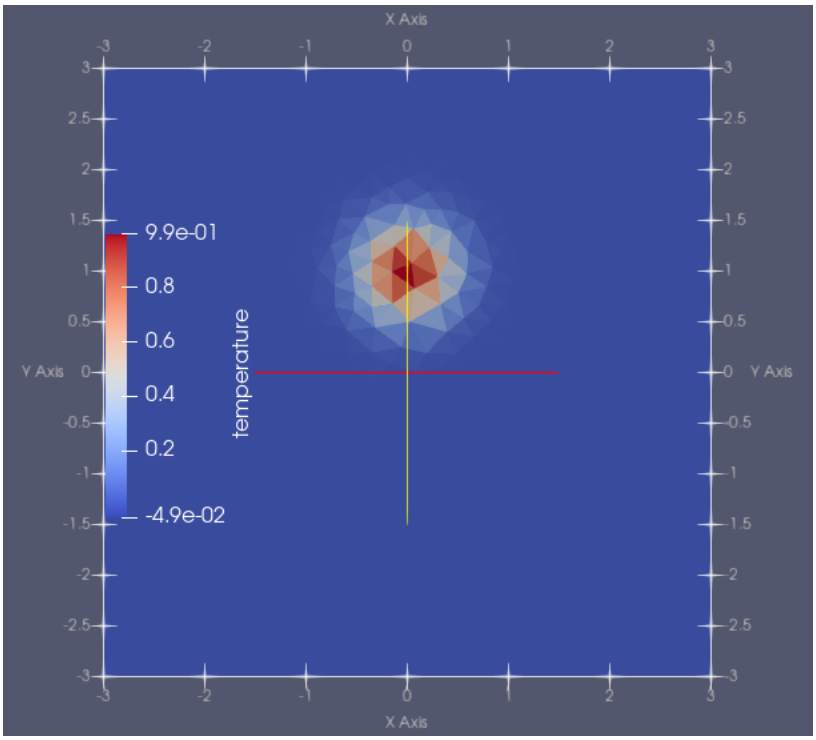


Figure 5: Initial Condition

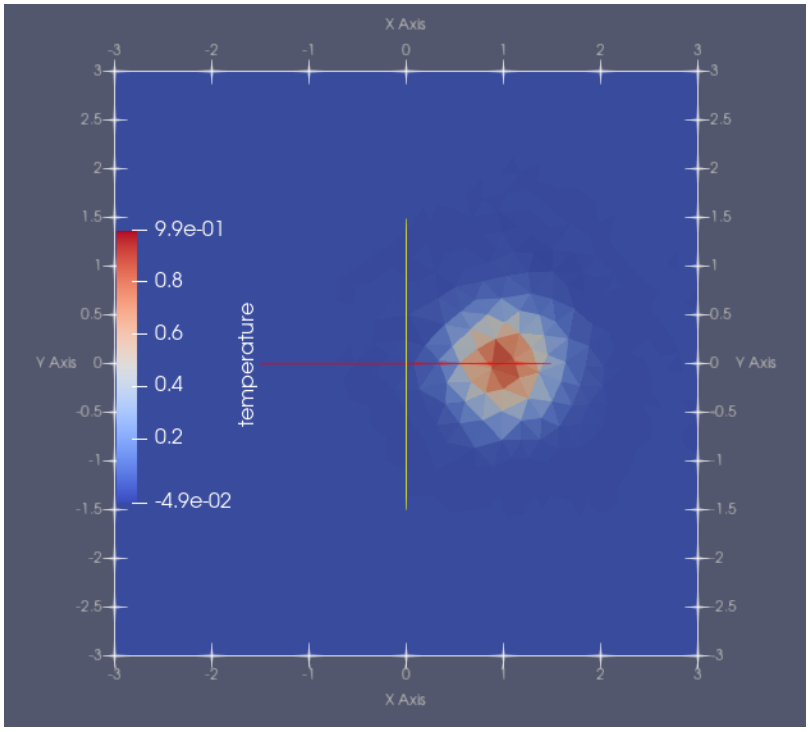


Figure 6: Temperature Field at T=0.5

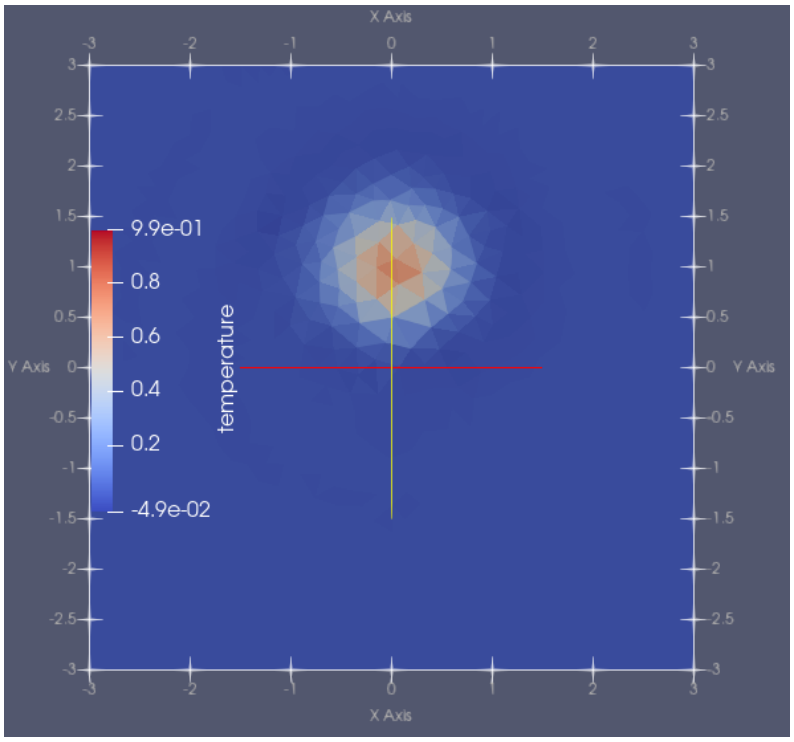


Figure 7: Temperature Field at T=2

Figure 5, Figure 6, and Figure 7 show the temperature field at T=0, 0.5, and 2 respectively, using the Adams-Bashforth time scheme. The expected behavior is that the initial data will rotate 90° clockwise

around the center of the domain by T=0.5, and it will have made a full 360° rotation after T=2. This expected behaviour matches what is seen. Similar results were found for the other time stepping schemes.

2. Describe the data structures that you will use to store data for this problem, especially connectivity data. Edge-based data structures are likely to prove most convenient but there are numerous other possibilities.

For this problem, edge based data structures were used. A class named Mesh was used to read the mesh, and store all of the edge connectivity data. The .mesh mesh files were used. These were selected, as they contained the most data about the edges of the mesh, having the cell on the left and right of each edge, as well as the initial and final vertex of each edge. Arrays and vectors from the c++ standard library were used to contain all of the data. These were used as they took care of all the memory management automatically. Using pointers and manually allocating and deallocating memory is another option, however this comes with the risk of memory leaks and other performance issues. With this knowledge at hand, the Mesh class contained the following member variables.

- Vert: Vert is an array containing 2 double vectors, each that had the length of the number of vertices found in the mesh file. This array stored the x and y coordinates of each vertex.
- Bdry: Bdry is an array containing 3 int vectors, each with length of the number of boundary edges. This array contained the indices of the cell adjacent to the edge, as well as the indices of the vertices that the edge contained. This array was not used very extensively, and can probably be deprecated.
- Edge: Edge is an array containing 4 int vectors, each with the length of the number of boundary edges. These vectors stored the indices of the left and right cells of each edge, as well as the indices of the vertices that make up the edge.
- Cell_edge: This is an array of 3 int vectors, each with the number of cells as the length. This array stores the indices of each edge for each cell.
- Cell_neighbor: This is an array of 3 int vectors, each with the number of cells as the length. This array stores the indices of each neighbor for each cell. In the case that the cell sits on the boundary it can have less than 3 neighbors. In this case, the value of -1 replaces the missing neighbor.
- Cell_vert: This is an array of 3 int vectors, each with the number of cells as the length. This array stores the indices of the vertices that make up the cell.
- Edge_length: This is a double vector with the number of edges as the length. This vector contains the edge length for each edge.
- Cell_area: This is a double vector with the number of cells as the length. This vector contains the area of each cell.
- Edge_norm: This is an array of 2 double vectors, each with the number of edges as their length. This array contains the x and y components of the normal vector of each edge, assuming the normal points to the right of the edge, as you walk from the edge origin to the edge destination vertex.
- Cell_centroid: This is an array of 2 double vectors, each with the length of the number of cells. This array contains the x and y coordinates of each cell centroid.
- Edge_centroid: This is an array of 2 double vectors, each with the length of the number of edges. This array contains the x and y coordinates of each edge midpoint.
- Edge_upwind: This is an array of 2 int vectors, each with the length of the number of edges. This array contains the indices of the upwind, and the downwind cell of each edge. This array only has to be calculated once in this case, as the velocity vector does not change.

- Dot_product: This is a double vector with the number of edges as the length. This vector stores the value of the dot product of the normal, and the velocity at each edge midpoint, which is needed for flux calculations, and determining the upwind cell for each edge.

The Mesh class also contained the following member functions, which can be used to populate the majority of the above arrays.

```
void calc_cell_edge();
void calc_cell_neighbor();
void calc_cell_vert();
void calc_edge_length();
void calc_cell_centroid();
void calc_edge_centroid();
void calc_edge_norm();
void calc_all_param();
```

Not all of them were populated upon reading the mesh, as they varied based on the initial condition. For example, the upwind cell to each edge, and the dot product of each edge normal and the velocity could not be calculated upon reading the mesh, as these components were dependant upon the initial conditions. All of the geometric quantities were calculated upon reading the mesh to save on runtime however. Finally ,a function read_mesh was created. This function initialized the Mesh object, and pulled all of the values of the .mesh file to store in the arrays described above. It also performed the cell connectivity functions described above, to minimize the amount of pre-processing required by the user.

For the solution, a variety of functions can be found in the field_functions.cc file. These functions include setting the initial condition, calculating cell gradients, upwind cells, edge fluxes, cell flux integrals, and L2Norms of vectors. The function to save vtk files, as well as the time stepping schemes can also be found here.

3. Write and validate a code that solves the convection equation to second order accuracy in both time and space. Basically, you will implement the algorithm you described in the first homework assignment. You should choose an appropriate explicit time advance scheme.

This problem was first validated using the problem described in the homework assignment. Once this was completed, the final problem was utilized for all validation. For time schemes, 3 second order, and 1 first order was used. These included, first order explicit forward Euler, second order Adams-Bashforth, second order midpoint method, and finally second order Runge-Kutta. Unexpected behaviour was encountered in the testing and validation of this code. The code worked as expected for some meshes, but for others the code faced instability. Even dropping the timestep as low as 5e-7 seen failure for the very fine mesh. What was most interesting, is that instabilities were also seen in the medium, and coarse meshes. In one of the experiments, the fine mesh was tested with a timestep of 1e-3, and the medium mesh was tested with a timestep of 1e-5. The fine mesh worked as expected, but the medium mesh faced instabilities, regardless of time stepping scheme, and timestep. The veryfine, medium, and coarse meshes all faced these instabilities. The instabilities showed a routine pattern as well, always occurring ~T=0.7, and starting from the same position in the domain.

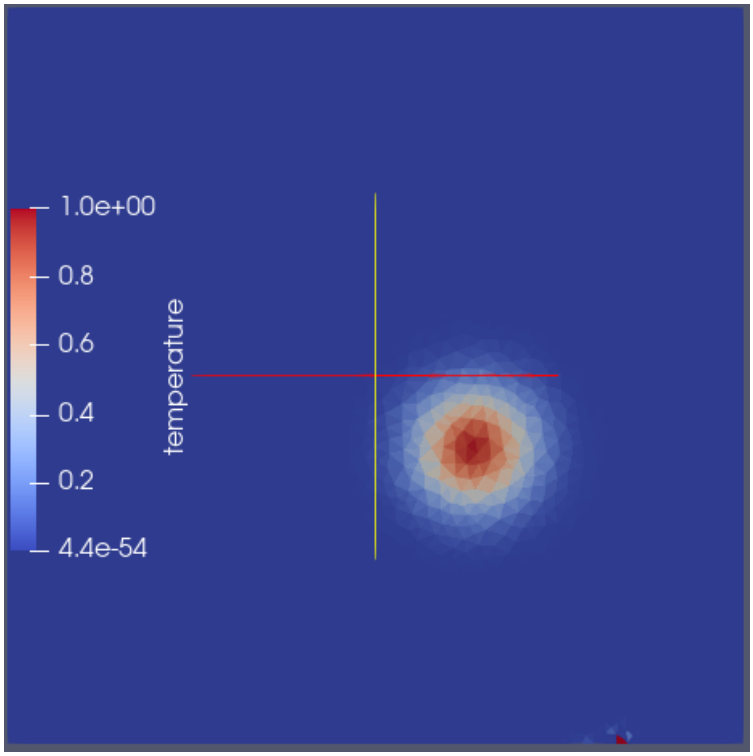


Figure 8: Instability Onset at T=0.7 for Veryfine Mesh Using RK2 Time Stepping

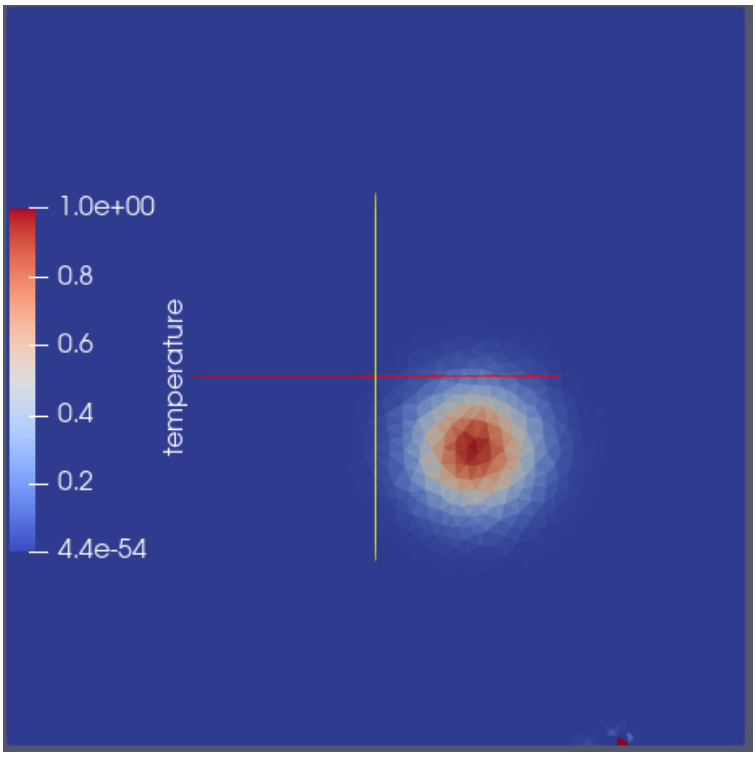


Figure 9: Instability Onset at T=0.7 for Veryfine Mesh Using Midpoint Time Stepping

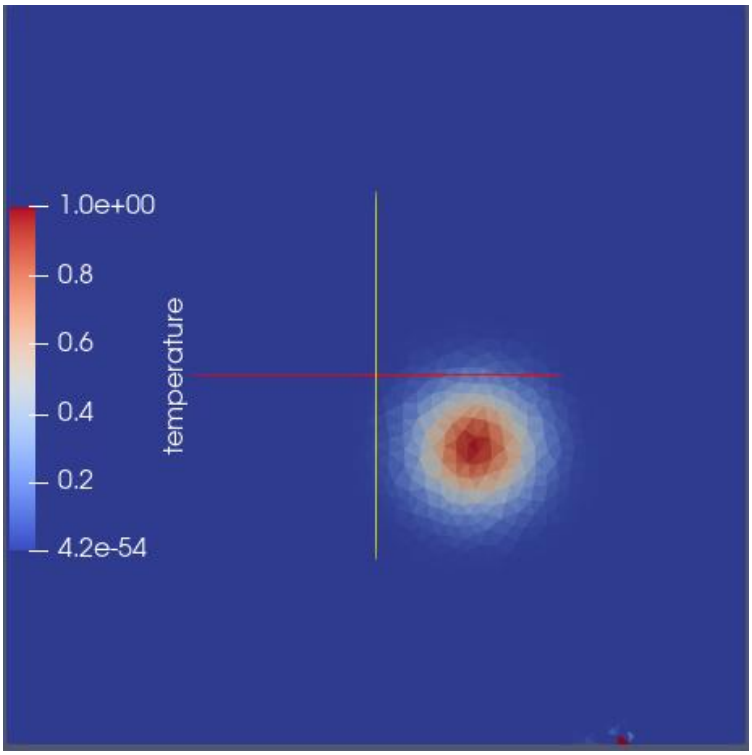


Figure 10: Instability Onset at T=0.7 for Veryfine Mesh Using Adam-Bashforth Time Stepping

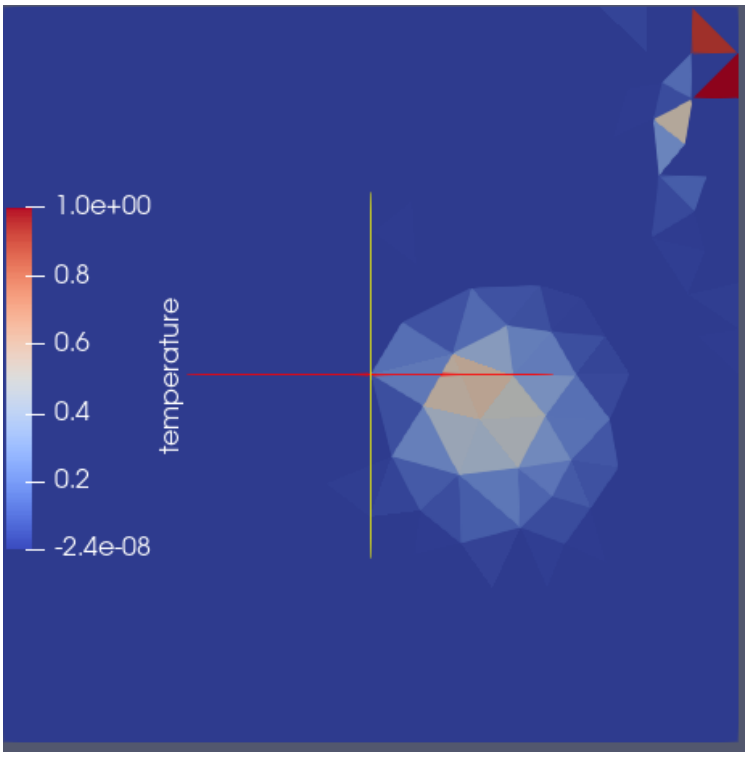


Figure 11: Instability Onset at T=0.6 for Medium Mesh Using RK2 Time Stepping

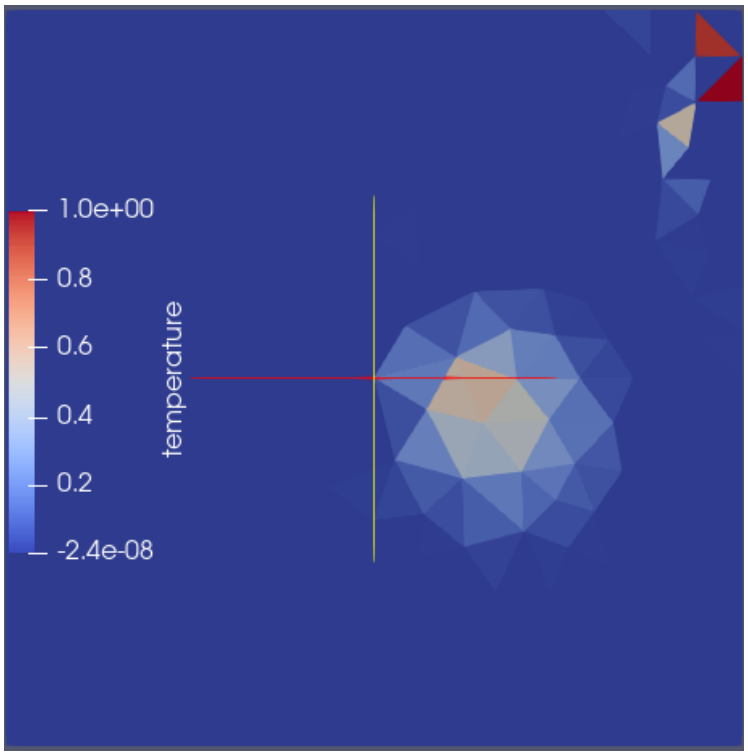


Figure 12: Instability Onset at T=0.6 for Medium Mesh Using Midpoint Time Stepping

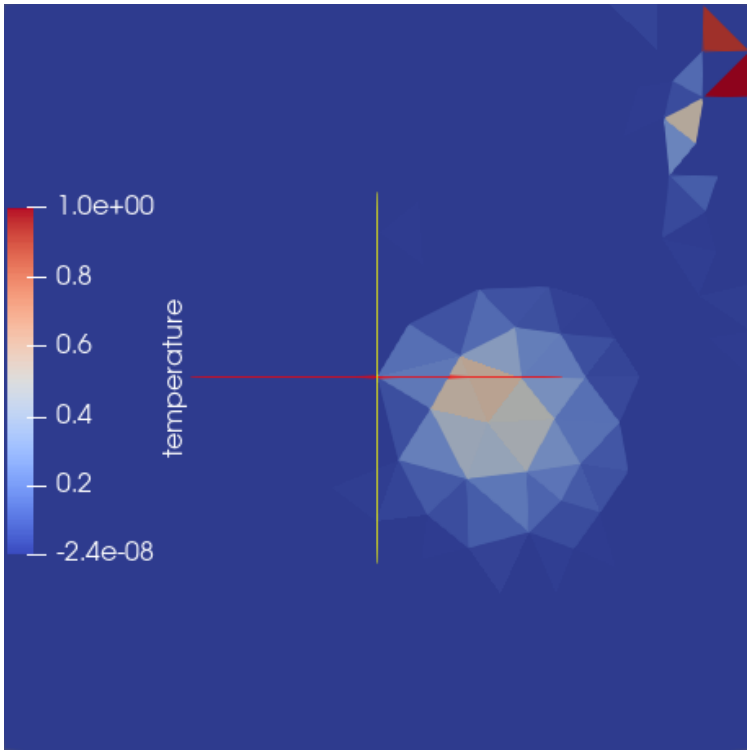


Figure 13: Instability Onset at T=0.6 for Medium Mesh Using Adams-Bashforth Time Stepping

The above figures show the instability onset for the veryfine and medium meshes with a timestep of 1e-4. Similar behavior was seen, regardless of the timestep for these meshes. As such, the fine and verycoarse meshes were used to determine the order of accuracy for these solutions. The coarse mesh exhibited weird behavior, but it was less regular than the veryfine and medium meshes. The reasoning for these strange behaviors is unknown, and can most likely be attributed to minor bugs in the code. The following graph shows the L2Norm Vs. grid size for each time scheme.

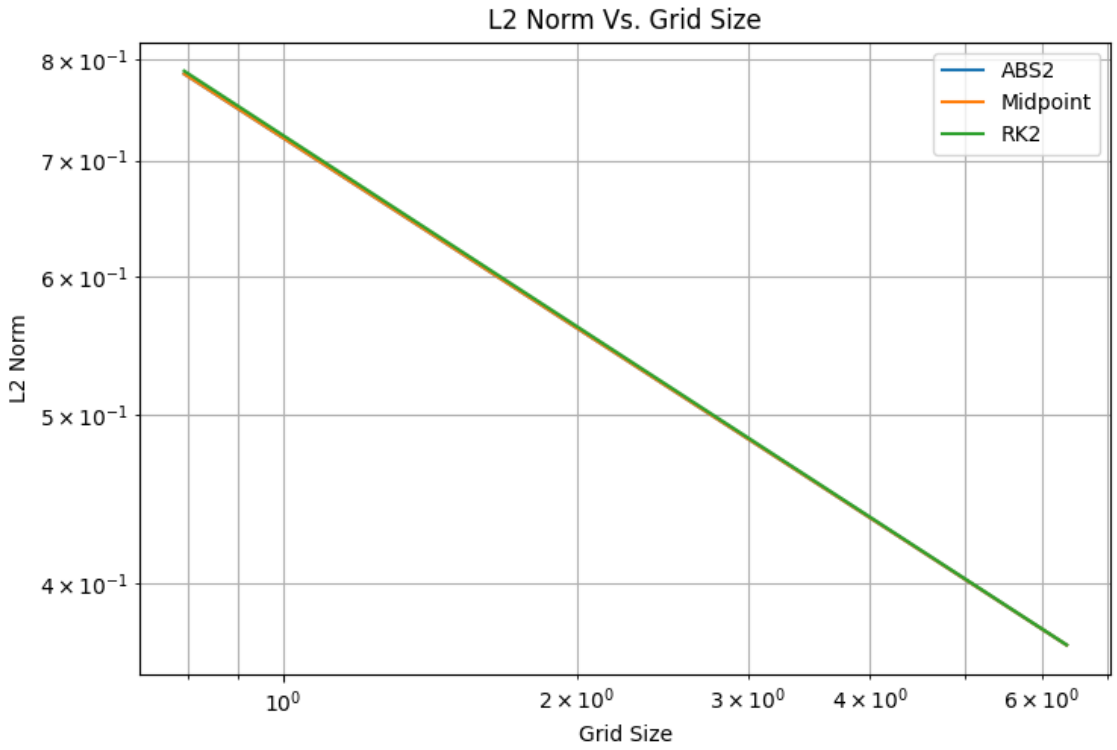


Figure 14: Timescheme Order of Accuracy

This figure shows a slope of -0.36 for each time scheme. This is off, with an expected slope of 1 (h is equivalent to area/sqrt(cells) as we are working in 2D, therefore a 2nd order accurate plot should show a slope of 1). The reasoning for this is unknown, and is most likely attributed to the same bugs that are demonstrated in the instability previously shown. The most suspected reasoning for these bugs would be the time stepping schemes, as these were not tested very rigorously. The remainder of the code was tested, however there is definitely room for improvement in the robustness of the tests. There are also a large number of areas where the code can be improved. For example, in the future, separate classes for cell, and edge data can be used. This can allow for easier accessing of connected cells, and other important data. A function to calculate the maximum timestep can also be implemented, to help with stability. Finally, a more rigorous testing plan can be implemented, which allows for increased robustness.

All mesh files, test files, and code files will be included in a zip folder with the submission for this code. In order to run the tests similar to those found in this report, the following variables can be changed.

```
std::string meshname = "Face-Cell/mech511-square-verycoarse.mesh"; // Enter mesh file name

double dt_max = 8.04e-3; // Enter maximum stable timestep here

double max_time = 2; // Enter maximum time here

double t0 = 0.0; // Enter initial time here

double CFL = 1.0; // Enter CFL number here. Timestep will be CFL*dt_max

int num_fps = 20; // How many solutions to save

int num_sol = max_time / (dt_max * num_fps); // How often do we want to save a solution file? Every num_sol timesteps, a solution will be saved

double dt = CFL * dt_max; // Timestep dt

int num_timestep = max_time / dt; // Number of timesteps before solution finishes

/*

Below, select the time scheme

1 - Euler

2 - Adams Bashforth

3 - Midpoint

4 - RK2

*/

int timescheme = 4;
```

For meshname, just change the “verycoarse” section, to match with the desired mesh. dt_max represents the maximum stable timestep. This variable was included to allow for easier integration of a function which calculates the maximum stable timestep. max_time represents the final time that the simulation will approach. t0 represents the initial time, which in the case of this simulation will always be 0. CFL represents the CFL number. As the maximum stable timestep is unknown, this can be left at 1. num_fps represents how many solutions will be saved for the simulation. In the case that you wish to save a solution every x timesteps, you can comment this out, and change num_sol to be x. Finally, timescheme allows for easy selection of the timescheme. Enter your desired timescheme and run. A makefile will be included as well. make run will perform all the compilation to allow for running the file. make all will compile every file. make clean will remove all object files, as well as run files. When running this code, all the output solutions will be placed into the results folder, saved under the name Timestep-X.vtk, where x represents the current timestep. This format is used, as Paraview automatically recognizes that each file represents a different timestep of the simulation, and will allow for running animations of the simulation. Finally, the following messages will be printed in the command line while running the code.

- If the time scheme is improperly selected, the code will inform you, and print the possible choices. The code will then exit.
- The code will inform the user whenever a VTK file is saved, every num_sol timesteps.

- Once the code reaches the last timestep, it will store the temperature field, to be used for error calculations later. The code will inform the user when this occurs.
- If the L2Norm ratio begins to explode, the code will warn the user. Once the L2norm ratio increases above 5, the code will begin printing that the solution may be exploding. This allows for easy monitoring of multiple simulations at once.
- At every timestep, the code will inform the user of which timestep it is at, the current time, the timestep interval dt, and the time marching scheme used. It will also inform the user of the L2Norm of the current timestep, and the L2Norm ratio at the current timestep.
- Once the time stepping is complete, the code will inform the user how many timesteps were performed, as well as which time scheme was used.
- The code will print the final L2Norm error, as well as the time that the code took to run, in seconds.